



FLASK TUTORIAL

By Ahmed Kiel-Kamil

3 PARTS OF WEB APP

- Frontend: the visual interface of an application that allows the user to interact with the functionality of the application
- Backend: the business logic that is responsible for communicating with the infrastructure to achieve the specific function of the application
- Infrastructure: The external resources and or programs needed to complete the tasks expected from the application.
 - For the task of data storage we use a database

Question: what software technique do we use to allow these programs to communicate

EXAMPLE

Welcome Back **Kobe Hulett**

What Can I Help With Today?



Enter Question Here

Generate

Frontend

```
def callAI(prompt):  
    api_url = AZURE_OPENAI_ENDPOINT  
    headers = {  
        "Content-Type": "application/json",  
        "api-key": AZURE_OPENAI_API_KEY,  
    }  
    data = {  
        "messages": [  
            {"role": "system", "content": "Hi I am your AI assistant"},  
            {"role": "user", "content": prompt}  
        ]  
    }  
    try:  
        response = requests.post(api_url, headers=headers, json=data, timeout=12)  
        if response.status_code == 200:  
            api_response = response.json()  
            content = api_response['choices'][0]['message']['content']  
            if not isinstance(content, str):  
                content = str(content)  
            return content  
        else:  
            return f"Bad call response code {response.status_code}"  
    except requests.RequestException as e:  
        last_err = RuntimeError(f"Request to Azure OpenAI failed: {e}")  
        return last_err
```

Backend

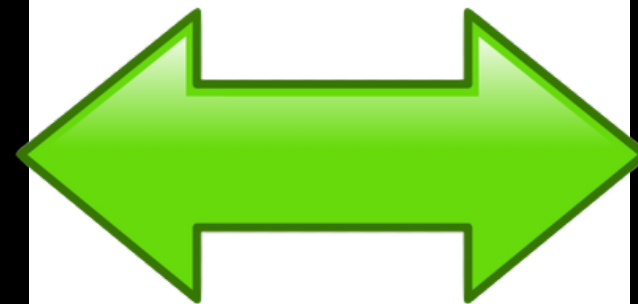


Infrastructure

WHAT IS FLASK

A python library that makes python functions available via and HTTP API call

```
fetch("http://localhost:5000/addContext", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({name: "Supreme"})
})
.then(response => response.json())
.then(data => {
  meep = document.getElementById("result")
  meep.innerHTML = data.content
})
.catch(error => {
  console.error("POST error:", error);
});
}
```



```
from flask import Flask, jsonify
#configures network infrastructure for this program
app = Flask(__name__)
#marks gives this function a name/accesible route
@app.route('/addContext', methods=['POST'])
def make_sentence():
  #returns the JSON within the request as a dictionary
  data = request.get_json()
  #retrieves the value held within the name member
  promptP = data["name"]
  #adds mor context to that string
  sentence = f"My name is {promptP}"
  #returns you value with the other request components
  return jsonify({"content":sentence})
#makes this a continously running process on a port
if __name__ == '__main__':
  app.run(debug=True)
```

FLASK X HTML

Flask has an additional function for user to incorporate HTML files called render template

Step 1: A function that recreates your html file by adding your code to a text file with an existing html template. Then sends that file as a string to the browser who is in charge of parsing you html

- The engine used to complete process is called Jinja 2

Step 2: This consolidated the frontend to the same internal address (port) as the rest of the running program (process)

Step 3: This allows all the HTML scripts to be accessible as different functions within one address

Step 4: This tightly coupled process configuration shortens your API endpoints to be just the name function

DEMONSTRATION TIME!

✓ Templates

<> index.html

```
9  @app.route('/index')
10  def index():
11      return render_template('index.html')
```

```
fetch("http://localhost:5000/addContext",
```



```
fetch("/addContext",
```

URL: `http://localhost:5000/index`

HOUSE KEEPING

You will have to initialize a virtual environment within your director in order to use the package manager pip that contains flask

1. `python3 -m venv venv`
2. `source venv/bin/activate` (for mac) `.\venv\Scripts\Activate.ps1` (for Windows)
3. `pip install flask`

If your Flask server needs to be accessible to uncouple frontend programs, you will need to include the CORS library as well

```
from flask_cors import CORS  
app = Flask(__name__)  
CORS(app)
```

Be careful about any network traffic or concurrent processes surrounding the port you are using as they will interfere. Ex. VPN's browser extensions

BENEFITS OF FLASK

- Networking abstraction
- Consolidating process
- Improved infrastructure security
 - Look into environment variables before deployment
- HTML scripts gets access to python libraries and other infrastructure

```
from transformers import pipeline
from azure.cosmos import CosmosClient, exceptions
from azure.storage.blob import ContainerClient
```




THANK YOU